



ESCUELA POLITÉCNICA NACIONAL
FACULTAD DE INGENIERÍA DE SISTEMAS
APLICACIONES MÓVILES

Nombre: Erick Erazo, Luis Guerrero, Gabriel Maldonado

Curso: GR1SW

Fecha: 15/04/2026

Taller en Clase 01

1. Uber ha mantenido apps nativas (Swift / Kotlin) para su producto principal mientras usa RIBs, su propio framework de arquitectura. ¿Por qué una empresa con los recursos de Uber no migró a Flutter o React Native para unificar sus equipos de iOS y Android?

Uber no migró a Flutter o React Native porque su problema no era solo compartir UI, sino escalar una arquitectura altamente modular, resiliente y desacoplada como RIBs, optimizada para equipos grandes trabajando en paralelo; el costo de reescribir features críticas, la dependencia de APIs nativas (GPS en tiempo real, background services, notificaciones de baja latencia) y la necesidad de control fino sobre rendimiento y memoria hacen que el beneficio de unificar código no compense el riesgo técnico ni el impacto en velocidad.

2. Amazon ha mantenido apps nativas separadas para iOS y Android. Con un equipo de ingeniería de miles de personas, ¿por qué no unificaron el codebase con Flutter o React Native? ¿Qué factor de negocio o técnico lo hace más difícil para Amazon que para, digamos, una startup?

En Amazon, el desafío no es la duplicación de código sino la complejidad del dominio: múltiples verticales (retail, video, Alexa, pagos), integraciones profundas con hardware y servicios nativos, y ciclos de release altamente optimizados; migrar a un framework cross-platform introduciría un “lowest common denominator” que limita optimizaciones específicas por plataforma, y el costo de coordinación y re-arquitectura a escala organizacional es mucho mayor que en una startup con un producto más acotado.

3. Uber usa notificaciones push de alta frecuencia: estado del viaje, posición del conductor, cambios de precio dinámico. Si la app estuviera en Flutter, ¿el motor de renderizado propio (Impeller) sería una ventaja o desventaja para manejar actualizaciones de estado tan frecuentes desde el sistema operativo?

En un escenario como Uber, el motor Impeller de Flutter no es el cuello de botella principal; el problema real es la latencia y frecuencia de sincronización entre el estado nativo (ubicación, red, notificaciones) y la capa de UI, por lo que añadir una capa intermedia (Dart/engine) puede introducir overhead en el pipeline de actualización, haciendo que Flutter sea potencialmente una desventaja frente a UI nativa optimizada directamente sobre los callbacks del sistema operativo.



ESCUELA POLITÉCNICA NACIONAL
FACULTAD DE INGENIERÍA DE SISTEMAS
APLICACIONES MÓVILES

4. Amazon integra Alexa directamente en su app móvil: escucha voz, procesa lenguaje y responde. El procesamiento de audio en tiempo real tiene requerimientos de latencia de milisegundos. ¿Qué limitaciones arquitectónicas tendría React Native para manejar audio continuo en el hilo de JavaScript?

React Native tendría limitaciones claras para audio en tiempo real porque el hilo de JavaScript no está diseñado para procesamiento continuo de baja latencia; la comunicación entre el bridge (o JSI) y los módulos nativos introduce jitter y posibles bloqueos, lo que obliga a delegar casi todo el procesamiento a código nativo, reduciendo significativamente el valor de usar React Native en features como Alexa.

5. Si Uber lanza una experiencia en Apple Vision Pro donde el conductor aparece en tu campo visual con su ruta superpuesta en AR, ¿qué partes del stack actual de Uber (nativo, GPS, mapas) son portables a visionOS y cuáles tendrían que reescribirse desde cero?

Para una experiencia en visionOS, Uber podría reutilizar componentes como lógica de negocio, servicios de backend, modelos de datos, algoritmos de routing y consumo de APIs de mapas, pero tendría que reescribir completamente la capa de presentación, interacción y renderizado (UI/UX espacial, ARKit/RealityKit), así como adaptar la gestión de inputs y contexto espacial, ya que el paradigma de interacción en AR difiere radicalmente del móvil tradicional.

6. Amazon ya vende Echo Show (pantalla inteligente) y Fire TV. Si el próximo paso es una app de Amazon en un dispositivo plegable Samsung Galaxy Z Fold, ¿qué desafío arquitectónico específico presenta la transición de pantalla pequeña → desplegada en términos de gestión de estado y layout adaptativo?

En dispositivos plegables como el Galaxy Z Fold, el principal desafío es manejar estados consistentes entre múltiples configuraciones de layout dinámico (pantalla compacta vs expandida) sin reinicializar vistas ni perder contexto; esto exige una arquitectura de estado desacoplada de la UI (por ejemplo, unidirectional data flow) y layouts verdaderamente responsivos que soporten recomposición continua, algo más complejo que en móviles tradicionales donde el viewport es relativamente estático.